

The Complexity of Explaining Neural Networks Through (group) Invariants

Scott Neville,

Danielle Ensign,

Suresh Venkatasubramanian,

Arnab Paul

(University of Utah)

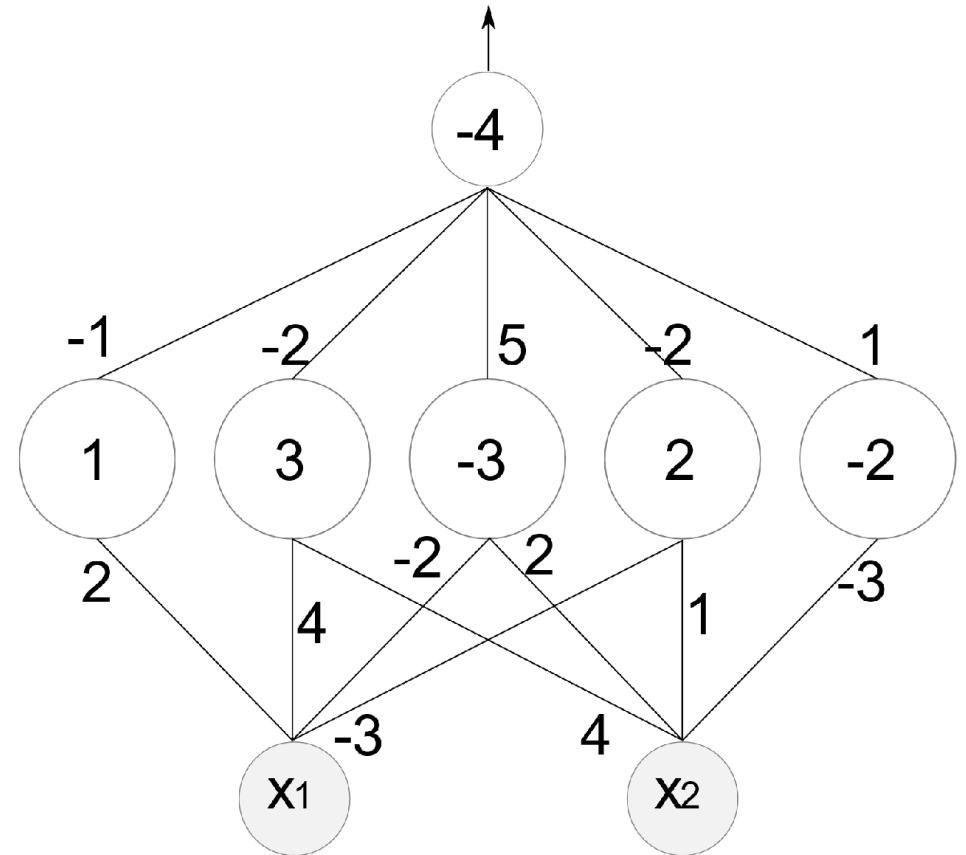
(Intel Labs)

Overview

1. What can models tell us about what they were trained on?
2. How can we ask this question for Neural Networks?
3. A proposed formulation
4. *Bad news*. Any version of this is hard.

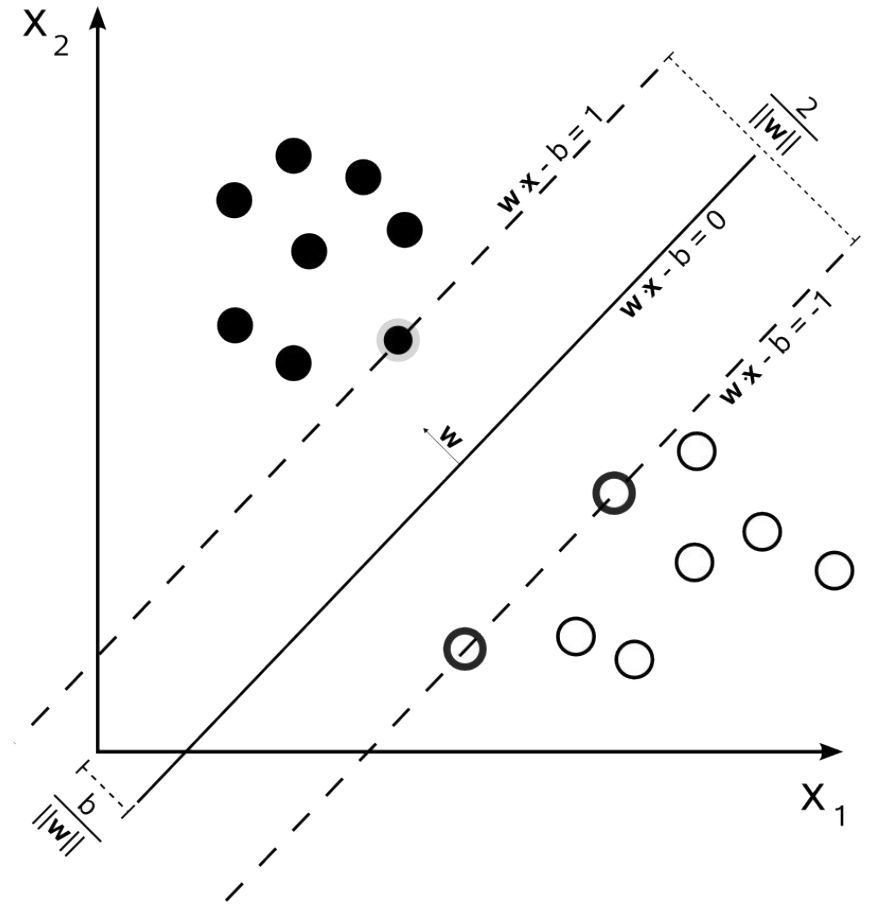
Late Night Project

- If each neuron computes:
 - $\text{sgn}(w x + b)$
- Can you tell me anything about the task?



Other Methods: Perceptron

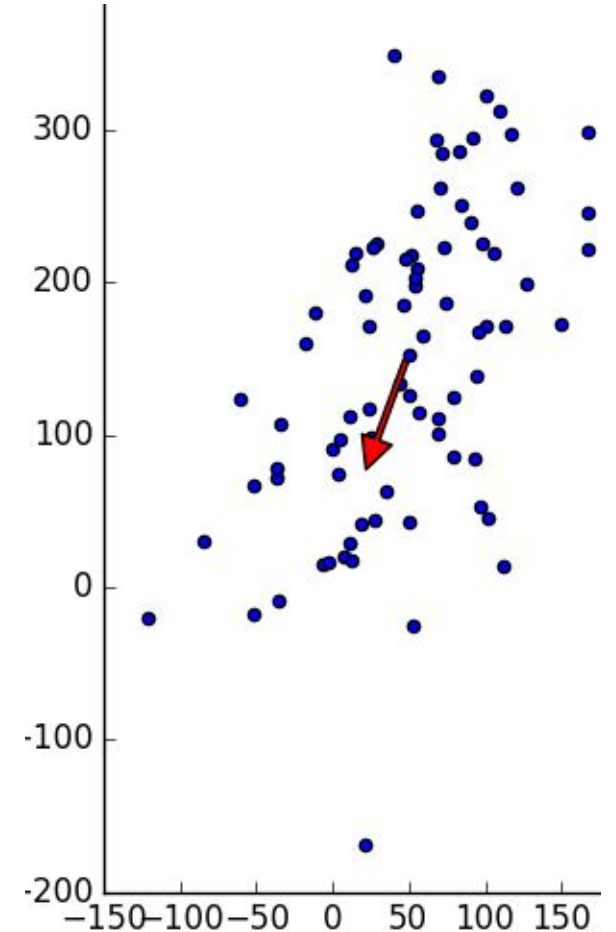
- An Perceptron immediately tells us a line of symmetry.
- Translate the dataset along this line keeps the labels the same.



By Cyc - Own work, Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=3566688>

Other Methods: PCA

- Given the PCA of a dataset we get 'ideal' coordinates for the dataset
- Ignoring less important coordinates gives a compression
- Movement along those coordinates shouldn't change the label

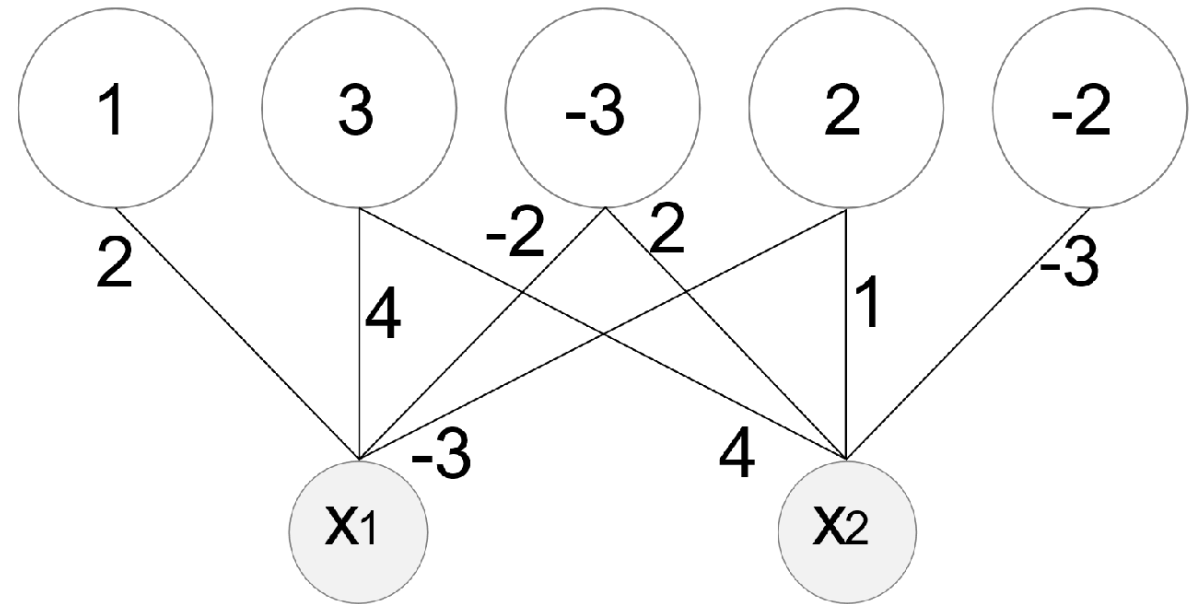


What we learn

- Invariant transformations of the data
 - Translations in the null space, or parallel to a hyperplane

A trained NN

- Gives us an invariant representation of the data
- What is it invariant to?



Function invariants

- A function NN is invariant to the function g if:
- For all inputs x : $NN(g(x)) = NN(x)$
- Example: $Sum(x,y) = Sum(y,x)$, so Sum is invariant to $f(x,y) = (y,x)$

G-sets

- The (invertible) invariants of a function form a **Group**!
 - Identity
 - Closed under Composition: $NN(f(g(x))) = NN(g(x)) = NN(x)$
 - Associative
- Groups that act on a set induce **Orbits**
 - $O(x) = \{s \mid f(x) = s, f \in INV(NN)\}$
- Every function can be written in two parts (Minsky)
 - Inputs to Orbits
 - Orbits to Labels

Encoded Invariants are Powerful

- Conv nets are invariant to small translations
- Gens and Domingos (2014) “Deep Symmetry Networks”
- Zaheer, et al. (2017) “Deep Sets”

How do they get there?

- Poggio, Anselmi (2014) “Unsupervised Learning of Invariant Representations with Low Sample Complexity...”
- Paul, Venkatasubramanian (2015) “Why does Unsupervised Deep Learning Work? – A Perspective from Group Theory”

Our Problem

- **FINDP**: Given a neural network NN, find a nontrivial invariant permutation of the inputs (or DNE if none exists).

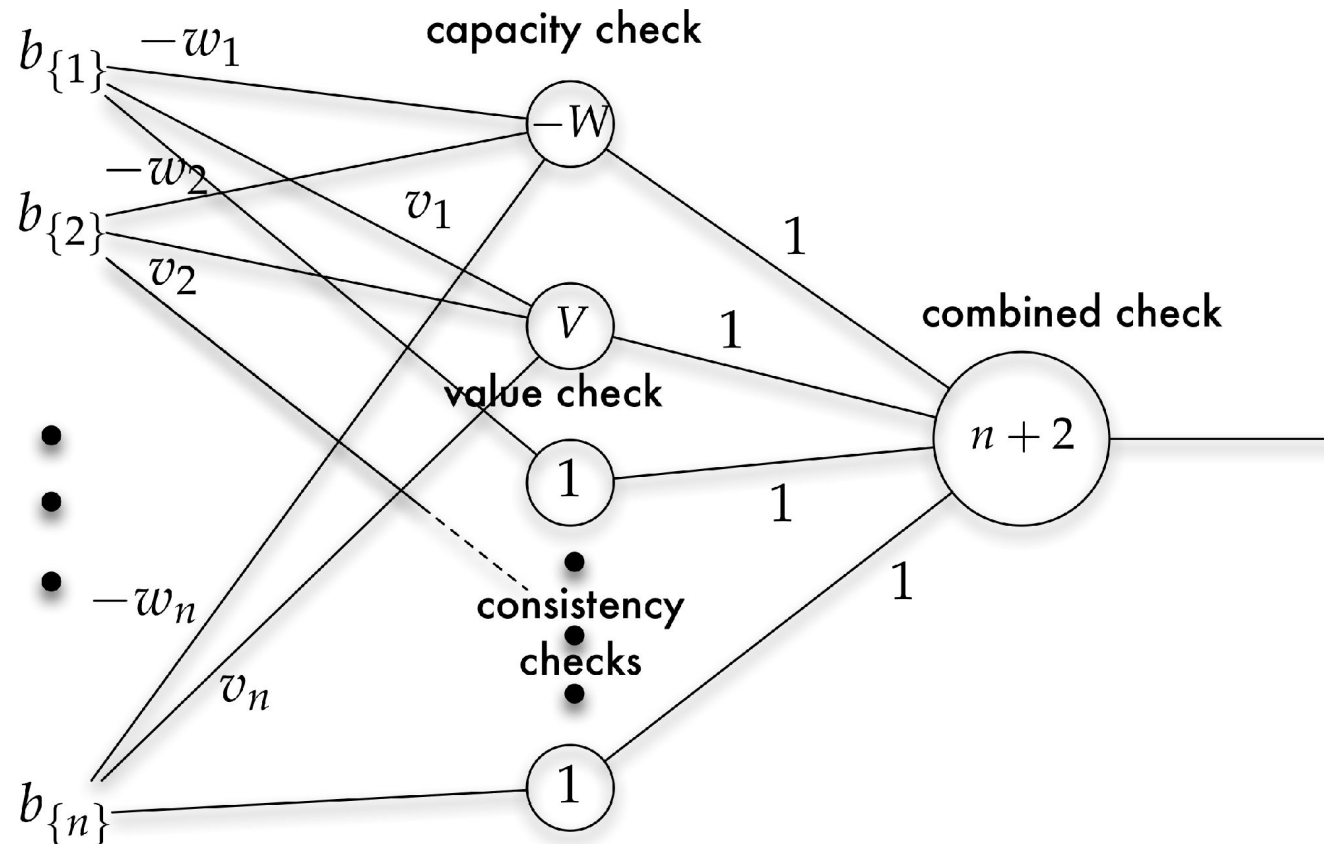
Our Main Result

FINDP is NP-Hard

- **Theorem:** It is NP-hard to find an invariant permutation of 2-layer Neural Network on Boolean inputs with the threshold activation function.

Proof Outline

- This problem Turing reduces from the 0,1 Knapsack Decision Problem.



What about...

- Real inputs?

NOPE.

- We can construct 'discrete recognizer' nodes, and use the proof as usual.

What about...

- Finding a $GL_n(R)$ invariant?

NOPE.

- Only permutations preserve the hypercube, so add the 'discrete recognizer' and the proof still works.

What about...

- Approximate Invariants?

NOPE.

- We can add dummy features so that each solution has exponentially many inputs.

What about...

- Continuous Activation Functions?

NOPE.

- Invariants are now epsilon invariants.
- We need only scale the weights like $1/\epsilon$ to make the discrete recognizer and proof work.

“Powerful” architectures are hard

- Theorem: FINDP is hard on Many other architectures:
 - General
 - Reasonably sized
 - Closed under Composition
- Essentially, if your learning architecture is powerful, efficient, and composable with itself, then understanding the result will be hard.

Future Work

Positive

- What about finding other Groups:
 - Only rotations, cycles, translations?
- What about Conv nets?

Negative

- Exactly how hard is FINDP?
- For other architectures, Kolomogorov's theorem is slightly too weak.

